

Implementation of an End-to-End Web Automation Testing Framework Using Playwright for OrangeHRM

Semester Project Report

Author: Minahil Murtaza

GitHub Repository: <https://github.com/minahil-murtaza/orangehrm-playwright-automation>

Abstract— Web applications have become an essential component of modern software systems, making software quality assurance a critical aspect of the development lifecycle. Manual testing of web applications is often repetitive, time-consuming, and susceptible to human error, particularly when validating frequently executed business workflows. Automated testing addresses these limitations by providing faster execution, improved accuracy, enhanced reusability, and better regression testing capabilities.

This project presents the design and implementation of an automated functional testing framework for the OrangeHRM Demo web application using Microsoft's Playwright framework with TypeScript. The framework adopts the Page Object Model (POM) design pattern to improve maintainability, modularity, and scalability while separating page interactions from test logic. Multiple functional modules, including Authentication, Dashboard, My Info, Personnel Information Management (PIM), and an End-to-End user workflow, were automated to validate core application functionality.

The implemented framework demonstrates structured project organization, reusable page components, centralized test data management, configurable execution settings, and automated validation of user interactions. The successful execution of the developed test suite confirms the effectiveness of Playwright for modern web application testing and highlights its suitability for developing reliable, maintainable, and scalable automation frameworks.

Keywords— Automation Testing, Playwright, TypeScript, Page Object Model, Functional Testing, End-to-End Testing, OrangeHRM, Web Testing

I. INTRODUCTION

Software testing is an essential part of the Software Development Life Cycle (SDLC). It helps identify defects, verify that the application works as expected, and ensures that the final product meets user requirements. Traditionally, many software applications are tested manually. Although manual testing is useful for exploratory and usability testing, it becomes repetitive, time-consuming, and prone to human error when the same test cases need to be executed repeatedly.

Automation testing addresses these challenges by allowing test cases to be executed automatically with greater speed, consistency, and accuracy. Automated tests can be reused, executed multiple times, and integrated into continuous development workflows, making them an important practice in modern software development.

Playwright is a modern open-source automation framework developed by Microsoft for testing web applications. It supports multiple browsers, provides reliable element handling, and offers powerful features such as automatic waiting, debugging tools, screenshots, and cross-browser testing. When combined with TypeScript, Playwright enables developers to build maintainable and scalable automation frameworks using structured programming practices.

This project focuses on automating the testing of the OrangeHRM Demo web application using Playwright with TypeScript. The framework follows the Page Object Model (POM) design pattern to separate page interactions from test logic, improving code readability, reusability, and maintainability. The implemented test suite covers major functional modules, including Authentication, Dashboard, My Info, Personnel Information Management (PIM), and an End-to-End workflow that simulates a complete user journey through the application.

The primary objective of this project is to design a structured automation testing framework that demonstrates industry-standard testing practices while validating the core functionalities of the OrangeHRM application. The developed framework serves as both a practical learning

experience in web automation testing and a reusable foundation for future enhancements.

II. RELATED TECHNOLOGIES

This project was developed using modern web automation technologies that support reliable, maintainable, and scalable test automation. The primary technologies used are described below.

A. Playwright

Playwright is an open-source web automation framework developed by Microsoft. It supports automation across Chromium, Firefox, and WebKit browsers using a single API. Playwright provides features such as automatic waiting, reliable element interaction, browser context isolation, and powerful debugging tools. These capabilities make it suitable for automating modern web applications with dynamic user interfaces.

B. TypeScript

TypeScript is a statically typed programming language built on JavaScript. It improves code quality by providing type checking, better error detection, and enhanced development support through modern editors. Using TypeScript in Playwright projects results in more readable, maintainable, and less error-prone automation scripts.

C. Page Object Model (POM)

The Page Object Model is a widely used design pattern in automation testing. It separates the page elements and user actions from the test cases by storing them in dedicated page classes. This approach improves code reusability, simplifies maintenance, and reduces duplication across the automation framework. In this project, each application module is represented by its own page object class, while the test scripts focus only on validating the expected behavior.

D. OrangeHRM

OrangeHRM is an open-source Human Resource Management System that provides various HR-related functionalities, including employee management, recruitment, leave management, attendance tracking, and personal information management. The publicly available OrangeHRM Demo application was selected as the system under test because it offers realistic business workflows suitable for functional automation testing.

E. Node.js and npm

Node.js provides the JavaScript runtime environment required to execute Playwright automation scripts, while npm (Node Package Manager) is used to install and manage project dependencies. Together, they simplify project setup, package management, and framework execution.

F. Visual Studio Code

Visual Studio Code was used as the primary Integrated Development Environment (IDE) for implementing the automation framework. Its support for TypeScript, debugging tools, Git integration, and Playwright extensions made the development and execution of automated test cases more efficient.

```
PS C:\Users\Hp\OneDrive\Desktop\orangehrm-playwright-automation> npx playwright --version
Need to install the following packages:
playwright@1.61.1
OK to proceed? (y) y
Version 1.61.1
PS C:\Users\Hp\OneDrive\Desktop\orangehrm-playwright-automation> npx playwright --version
Version 1.61.1
PS C:\Users\Hp\OneDrive\Desktop\orangehrm-playwright-automation> npm init playwright@latest

> orangehrm-playwright-automation@1.0.0 npx
> create-playwright

Getting started with writing end-to-end tests with Playwright:
Initializing project in '.'
√ Do you want to use TypeScript or JavaScript? · TypeScript
√ Where to put your end-to-end tests? · tests
√ Add a GitHub Actions workflow? (Y/n) · false
√ Install Playwright browsers (can be done manually via 'npx playwright install')? (Y/n) · true
Installing Playwright Test (npm install --save-dev @playwright/test)...

added 3 packages, and audited 4 packages in 8s

found 0 vulnerabilities
Installing Types (npm install --save-dev @types/node)...

added 2 packages, and audited 6 packages in 4s

found 0 vulnerabilities
Writing playwright.config.ts.
Writing tests/example.spec.ts.
Writing package.json.
```

Figure 1. Development Environment and Playwright Framework Setup

II. RELATED TECHNOLOGIES

The automation framework was designed to be modular, maintainable, and easy to extend by following the Page Object Model (POM) design pattern. The project separates page interactions, test scripts, test data, and utility functions into independent components, making the framework more organized and reusable.

The framework consists of five major components:

- **Page Objects:** Contains all web element locators and methods that perform actions on the application's pages. Each module of the OrangeHRM application is represented by a separate page class.
- **Test Scripts:** Contains the automated test cases that validate different functionalities of the application. These scripts use the methods defined in the Page Object classes instead of directly interacting with web elements.
- **Test Data:** Stores reusable test data such as login credentials, employee information, and profile details. Keeping test data separate from the test scripts improves maintainability and makes future updates easier.
- **Utilities:** Contains reusable helper functions that support the automation framework, such as capturing screenshots during test execution.
- **Configuration Files:** Includes the Playwright configuration file, TypeScript configuration, and package management files that define browser settings, execution behavior, project dependencies, and framework configuration.

The framework was implemented using the following project structure:

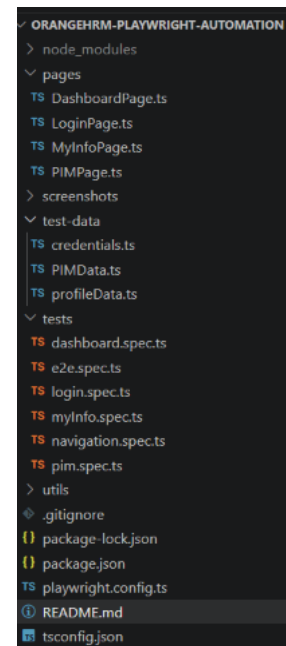


Figure 1. Development Environment and Playwright Framework Setup

The framework currently automates four major functional modules of the OrangeHRM application, namely Authentication, Dashboard, My Info, and Personnel Information Management (PIM). In addition, an End-to-End (E2E) test was implemented to validate the complete user workflow, starting from user authentication and ending with profile updates and logout.

This modular architecture allows new pages and test cases to be added with minimal modifications to the existing codebase, making the framework scalable for future enhancements.

III FRAMEWORK DESIGN AND IMPLEMENTATION

A. Page Object Model Design

The automation framework follows the **Page Object Model (POM)** design pattern, which is one of the most widely used approaches for developing maintainable automation frameworks. In this design, every web page or application module is represented by a separate class known as a *Page Object*. Each Page Object contains the web element locators and methods that perform user interactions on that page.

The test scripts do not directly interact with the web elements. Instead, they call the methods defined inside the Page Object classes. This separation of responsibilities improves code readability, reduces duplication, and simplifies framework maintenance. Whenever the user interface changes, only the corresponding Page Object needs to be updated while the test cases remain unchanged.

The developed framework contains four Page Object classes representing the major modules of the OrangeHRM application:

- **LoginPage.ts** – Handles user authentication by performing login operations and validating authentication-related scenarios.
- **DashboardPage.ts** – Provides methods for dashboard navigation, menu selection, and logout functionality.
- **MyInfoPage.ts** – Automates employee profile management, including updating personal information and validating successful updates.
- **PIMPage.ts** – Automates employee search operations within the Personnel Information Management module.

The relationship between the test scripts and the Page Object classes is illustrated in Figure 3.

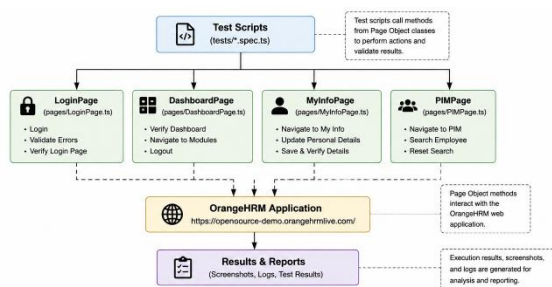


Figure 3: Interaction between Test Scripts and Page Object Classes

The following code snippet demonstrates the implementation of the login method inside the **LoginPage** class.

```
async login(username: string, password: string): Promise<void> {
```

```
    await
    this.usernameTextbox.fill(username);

    await
    this.passwordTextbox.fill(password);

    await this.loginButton.click();
}
```

The corresponding test case calls this reusable method instead of interacting directly with the web elements.

```
test('TC_Login_001 - Verify Successful Login', async ({ page }) => {

    await loginPage.login(

        credentials.validUsername,

        credentials.validPassword

    );

    await
    dashboardPage.verifyDashboardLoaded();

});
```

This implementation demonstrates the primary advantage of the Page Object Model. The login procedure is implemented only once inside the Page Object and can be reused across multiple test cases. As a result, the automation framework becomes easier to maintain, more organized, and less susceptible to code duplication.

B. Authentication Module

The Authentication module is responsible for validating the login functionality of the OrangeHRM web application. It verifies that authorized users can successfully access the system using valid credentials while preventing unauthorized access through invalid or incomplete login attempts. Since authentication is the entry point to the application, this module serves as the foundation for all subsequent automated test scenarios.

The implementation follows the Page Object Model (POM) design pattern. All page elements and user interactions are encapsulated within the `LoginPage` class, while the test scripts focus only on validating the expected behavior. This separation improves code readability, reduces duplication, and simplifies future maintenance.

1. LoginPage Implementation

The `LoginPage` class contains reusable methods that perform login operations and verify authentication-related functionality. The login process is implemented through a dedicated method that accepts the username and password as parameters.

```
async login(username: string, password: string): Promise<void> {  
  
    await  
    this.usernameTextbox.fill(username);  
  
    await  
    this.passwordTextbox.fill(password);  
  
    await this.loginButton.click();  
}
```

Similarly, validation methods are implemented to verify error messages, required field validations, and successful navigation after authentication. These reusable methods allow multiple test cases to share the same implementation without duplicating code.

2. Login Test Case Implementation

The test scripts interact only with the Page Object methods rather than directly locating web elements. The following example demonstrates the implementation of the successful login test case.

```
test('TC_Login_001 - Verify Successful Login', async ({ page }) => {  
  
    await loginPage.login(  
        credentials.validUsername,  
        credentials.validPassword  
    );  
});
```

```
        await  
        dashboardPage.verifyDashboardLoaded();  
    });  
});
```

Using reusable Page Object methods keeps the test scripts concise and easy to understand while improving maintainability.

3. Implemented Test Cases

The Authentication module contains seven automated test cases that validate both positive and negative login scenarios.

Test Case ID	Test Scenario	Status
TC_Login_001	Verify successful login using valid credentials	Passed
TC_Login_002	Verify login using an invalid username	Passed
TC_Login_003	Verify login using an invalid password	Passed
TC_Login_004	Verify login using invalid username and password	Passed
TC_Login_005	Verify login with an empty username	Passed
TC_Login_006	Verify login with an empty password	Passed
TC_Login_007	Verify login with empty username and password	Passed

The successful execution of the Authentication module is illustrated in **Figure 4**, where all implemented login scenarios were executed successfully using the Playwright automation framework.

```
PS C:\Users\jhp\OneDrive\Desktop\orangehrm-playwright-automation> npm run test:login  
> orangehrm-playwright-automation@1.0.0 test:login  
> playwright test tests/login.spec.ts  
  
Running 7 tests using 1 worker  
✓ 1 tests/login.spec.ts:44:9 › Authentication Module › TC_Login_001 - Verify Successful Login (14.8s)  
✓ 2 tests/login.spec.ts:55:9 › Authentication Module › TC_Login_002 - Verify Login with Invalid Username (11.2s)  
✓ 3 tests/login.spec.ts:78:9 › Authentication Module › TC_Login_003 - Verify Login with Invalid Password (9.8s)  
✓ 4 tests/login.spec.ts:91:9 › Authentication Module › TC_Login_004 - Verify Login with Invalid Credentials (9.4s)  
✓ 5 tests/login.spec.ts:104:9 › Authentication Module › TC_Login_005 - Verify Empty Username Validation (9.8s)  
✓ 6 tests/login.spec.ts:128:9 › Authentication Module › TC_Login_006 - Verify Empty Password Validation (9.8s)  
✓ 7 tests/login.spec.ts:136:9 › Authentication Module › TC_Login_007 - Verify Empty Credentials Validation (15.1s)  
  
7 passed (1.3m)
```

Figure 4. Successful Execution of the Authentication Module Test Cases.

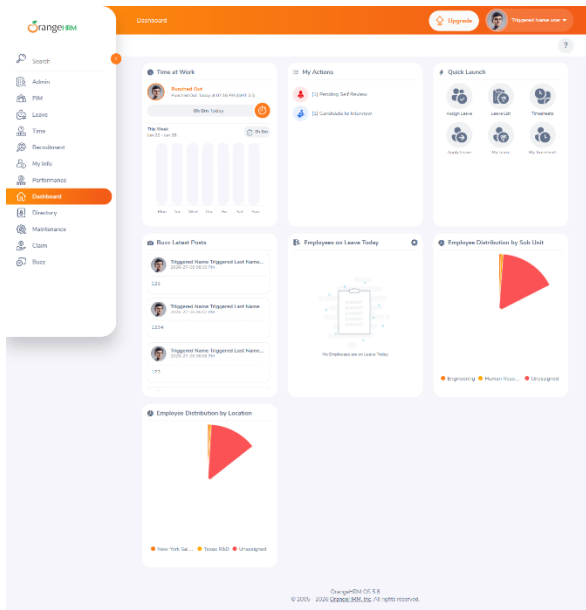


Figure 5. Successful Login Using Valid Credentials



Figure 6. Login Attempt with Invalid Username



Figure 7. Login Attempt with Invalid Password

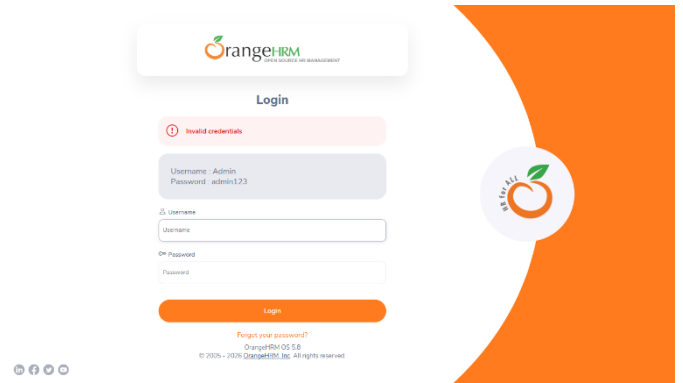


Figure 8. Login Attempt with Invalid Username and Password

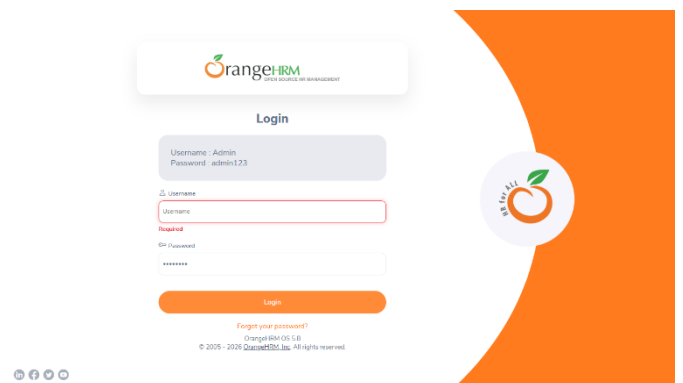


Figure 9. Validation for Empty Username

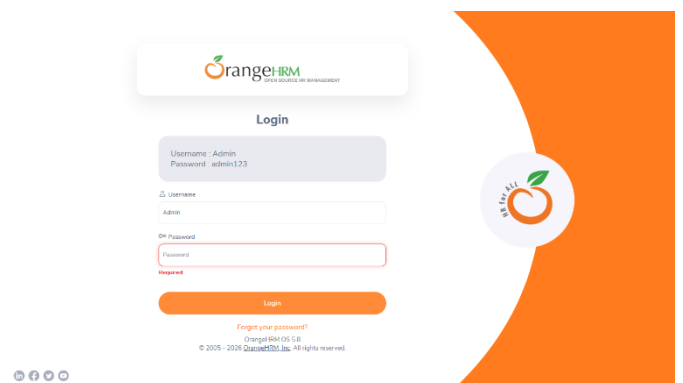


Figure 10. Validation for Empty Password



Figure 11. Validation for Empty Login Credentials

C. Dashboard Module

The Dashboard module verifies the successful navigation and accessibility of the OrangeHRM dashboard after user authentication. It ensures that authenticated users can access the main dashboard, navigate between application modules, interact with the user menu, and log out of the application successfully.

The implementation follows the Page Object Model by encapsulating all dashboard-related elements and user actions within the `DashboardPage` class. This approach separates page interactions from the test scripts, making the framework more organized and easier to maintain.

DashboardPage Implementation

The `DashboardPage` class provides reusable methods for verifying dashboard accessibility, navigating to different modules, and performing logout operations. The following method demonstrates the implementation used to verify that the dashboard has loaded successfully after authentication.

```
async verifyDashboardLoaded():
Promise<void> {

    await expect(this.dashboardHeading)
        .toBeVisible();

}
```

Navigation to application modules is also implemented through reusable methods.

```
async navigateToMyInfo(): Promise<void> {

    await this.myInfoMenu.click();

}
```

These methods allow multiple test cases to reuse the same implementation without directly interacting with page elements.

Dashboard Test Case Implementation

The following test case verifies that the user is successfully redirected to the dashboard after authentication.

```
test('TC_Dashboard_001 - Verify Dashboard
Navigation', async ({ page }) => {

    await
dashboardPage.verifyDashboardLoaded();

});
```

Additional test cases reuse the same Page Object methods to validate dashboard components, navigation to the My Info module, user menu functionality, and logout.

Implemented Test Cases

The Dashboard module consists of five automated test cases.

Test Case ID	Test Scenario	Status
TC_Dashboard_001	Verify dashboard navigation	Passed
TC_Dashboard_002	Verify dashboard components	Passed
TC_Dashboard_003	Verify My Info navigation	Passed
TC_Dashboard_004	Verify user dropdown menu	Passed
TC_Dashboard_005	Verify logout functionality	Passed

The Dashboard module executed successfully, and all test cases produced the expected results. Figure 5 presents the execution output of the dashboard automation test suite.

```
PS C:\Users\vip\OneDrive\Desktop\orangerm-playwright-automation> npm run test:dashboard
> orangerm playwright automation@1.0.0 test:dashboard
> playwright test tests/dashboard.spec.ts

Running 5 tests using 1 worker:

  ✓ 1 tests/dashboard.spec.ts:33:9 › Dashboard Module › TC_Dashboard_001 - Verify Dashboard Navigation (12.8s)
  ✓ 2 tests/dashboard.spec.ts:46:9 › Dashboard Module › TC_Dashboard_002 - Verify Dashboard Components (15.0s)
  ✓ 3 tests/dashboard.spec.ts:65:9 › Dashboard Module › TC_Dashboard_003 - Verify My Info Navigation (13.9s)
  ✓ 4 tests/dashboard.spec.ts:80:9 › Dashboard Module › TC_Dashboard_004 - Verify User Dropdown (22.3s)
  ✓ 5 tests/dashboard.spec.ts:95:9 › Dashboard Module › TC_Dashboard_005 - Verify Logout (12.9s)

5 passed (1.3m)
```

Figure 12. Successful Execution of Dashboard Module Test Cases.

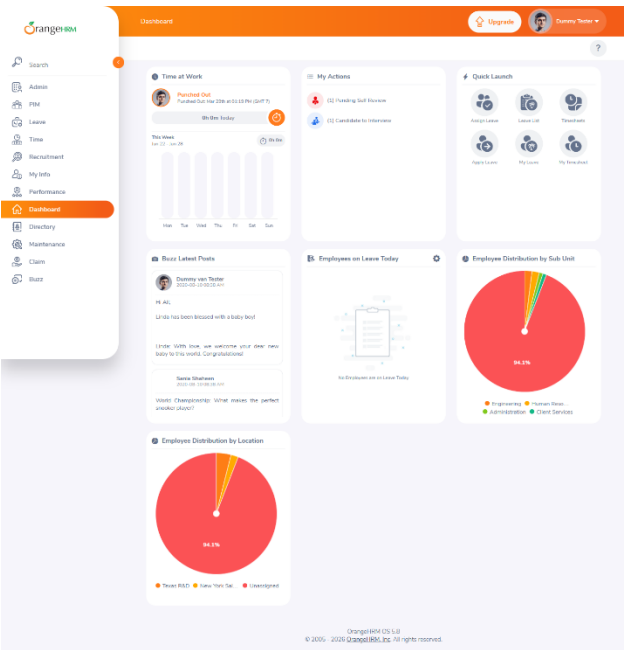


Figure 12. Successful Navigation to the Dashboard

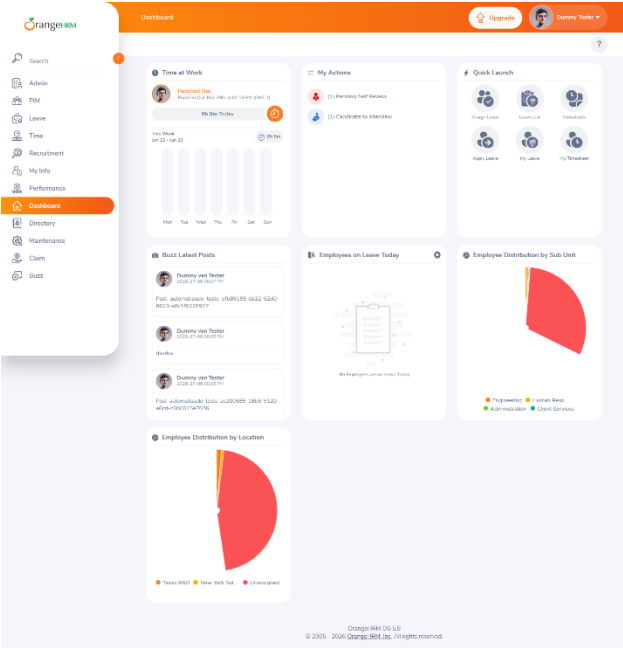


Figure 13. Verification of Dashboard Components

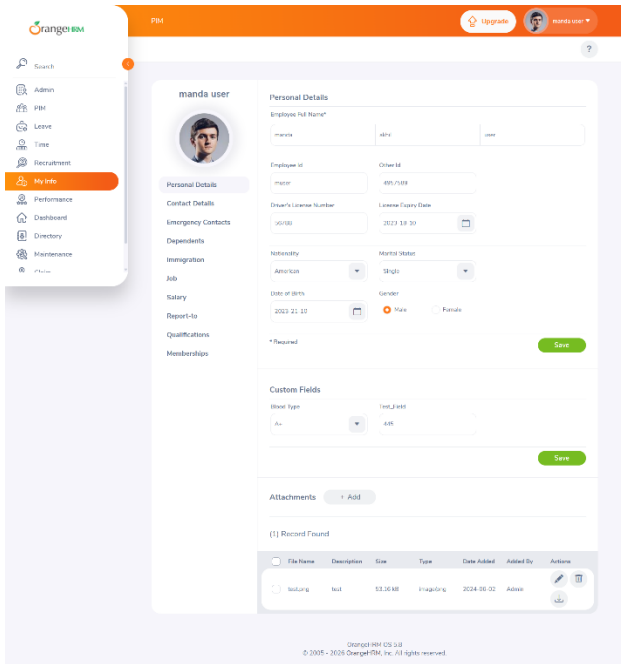


Figure 14. Navigation from Dashboard to the My Info Module

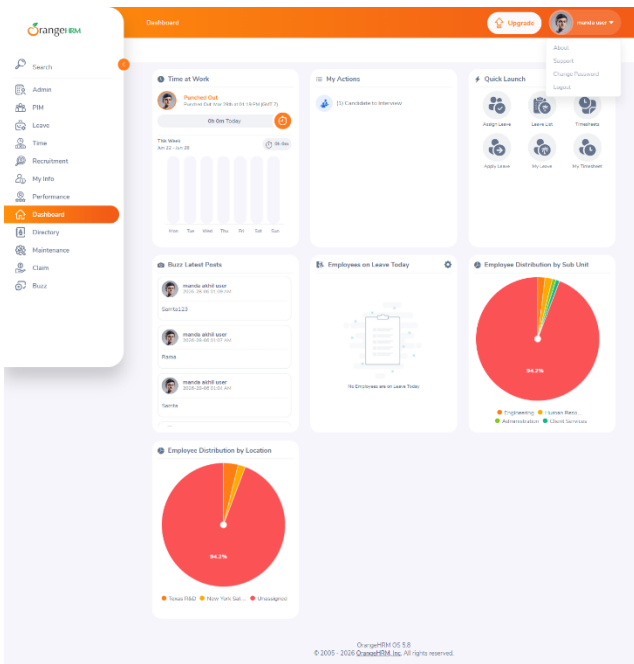


Figure 15. Verification of User Dropdown Menu

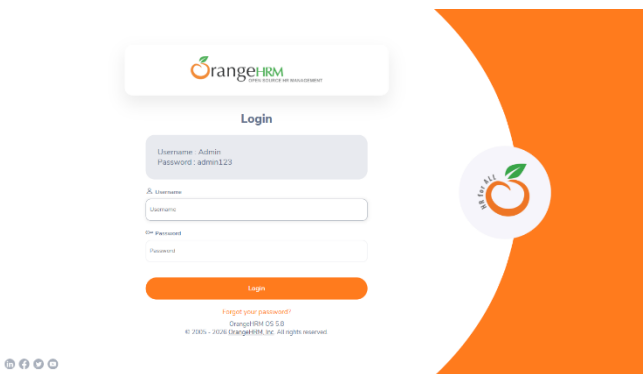


Figure 16. Successful User Logout

D. My Info Module

The My Info module validates the employee profile management functionality provided by the OrangeHRM application. It verifies that authenticated users can access their personal information, update profile details, save the changes successfully, and confirm that the updated information is reflected within the application.

The implementation follows the Page Object Model design pattern by encapsulating all profile-related web elements and user interactions inside the `MyInfoPage` class. This separation of

responsibilities improves code reusability, readability, and maintainability while allowing the test scripts to focus only on validating the expected application behavior.

MyInfoPage Implementation

The `MyInfoPage` class contains reusable methods for navigating to the employee profile, updating personal information, saving the modified data, and validating successful updates. The following method demonstrates the implementation used to update multiple personal information fields through a single reusable function.

```
async updatePersonalDetails(
    firstName: string,
    middleName: string,
    lastName: string,
    nickname: string,
    employeeId: string
): Promise<void> {

    await
    this.updateFirstName(firstName);

    await
    this.updateMiddleName(middleName);

    await this.updateLastName(lastName);

    await this.updateNickname(nickname);

    await
    this.updateEmployeeId(employeeId);
}
```

The profile information is saved using a dedicated method that performs the save operation independently of the test scripts.

```
async savePersonalDetails():
Promise<void> {

    await this.saveButton.click();
}
```

This implementation allows different test cases to reuse the same methods whenever employee information needs to be updated or validated.

My Info Test Case Implementation

The following test case demonstrates the automation of updating employee profile information.

```
test('TC_MyInfo_003 - Update Personal  
Details', async ({ page }) => {  
  
    await  
myInfoPage.updatePersonalDetails(  
  
        personalData.firstName,  
        personalData.middleName,  
        personalData.lastName,  
        personalData.nickname,  
        personalData.employeeId  
  
    );  
  
});
```

The remaining test cases reuse the same Page Object methods to verify navigation, profile accessibility, successful data saving, success message validation, and persistence of updated information.

Implemented Test Cases

The My Info module consists of six automated test cases.

Test Case ID	Test Scenario	Status
TC_MyInfo_001	Verify My Info page navigation	Passed
TC_MyInfo_002	Verify Personal Details page	Passed
TC_MyInfo_003	Update Personal Details	Passed
TC_MyInfo_004	Save Personal Details	Passed
TC_MyInfo_005	Verify success message after saving	Passed
TC_MyInfo_006	Verify updated information persists after refresh	Passed

The successful execution of the My Info module confirms that the automation framework can interact with editable profile information, perform update operations, validate successful data submission, and verify the expected system behavior. Figure 6 illustrates the successful execution of the implemented My Info test suite.

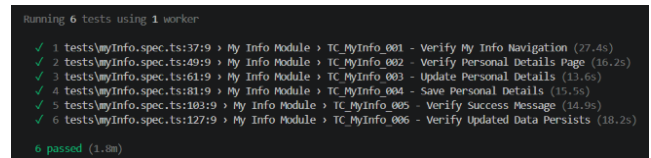


Figure 17. Successful Execution of My Info Module Test Cases.

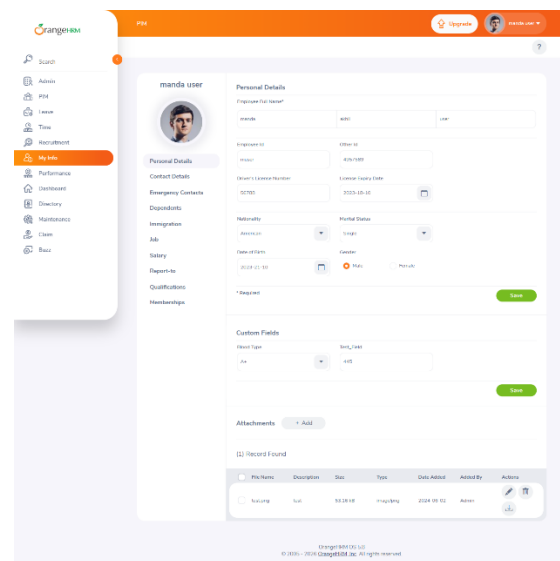


Figure 18. Successful Navigation to the My Info Module

Personal Details

Employee ID: 456789 | Other ID: 123456 | Name: John Doe | Address: 123 Main St, New York, NY 10001 | Country: USA

Contact Details: Email: john.doe@company.com | Phone: (123) 456-7890 | License Expiry Date: 2023-12-31

Emergency Contacts: Name: Jane Doe | Relationship: Spouse | Phone: (123) 456-7891

Dependents: Name: Alice | Relationship: Child | Date of Birth: 2005-01-15

Immigration: Nationality: American | Visa Status: Valid | Expiry Date: 2025-12-31

Job: Job Title: Software Engineer | Department: Engineering | Salary: \$75,000 | Report-to: Manager

Qualifications: Degree: Bachelor's in Computer Science | Institution: State University | Graduation Year: 2018

Memberships: Professional Association: IEEE | Membership Number: 123456789 | Expiry Date: 2024-06-30

Custom Fields: Field Type: Text | Value: 123456 | ID: 1 | Value: 789012

Attachments: + Add

Record Found

ID	Name	Description	Date	Type	Date Added	Added By	Action
1	Resume	Resume	2023-10-15	Document	2023-10-15	John Doe	Download

Figure 19. Updating Employee Personal Information

Personal Details

Employee ID: 456789 | Other ID: 123456 | Name: John Doe | Address: 123 Main St, New York, NY 10001 | Country: USA

Contact Details: Email: john.doe@company.com | Phone: (123) 456-7890 | License Expiry Date: 2023-12-31

Emergency Contacts: Name: Jane Doe | Relationship: Spouse | Phone: (123) 456-7891

Dependents: Name: Alice | Relationship: Child | Date of Birth: 2005-01-15

Immigration: Nationality: American | Visa Status: Valid | Expiry Date: 2025-12-31

Job: Job Title: Software Engineer | Department: Engineering | Salary: \$75,000 | Report-to: Manager

Qualifications: Degree: Bachelor's in Computer Science | Institution: State University | Graduation Year: 2018

Memberships: Professional Association: IEEE | Membership Number: 123456789 | Expiry Date: 2024-06-30

Custom Fields: Field Type: Text | Value: 123456 | ID: 1 | Value: 789012

Attachments: + Add

Record Found

ID	Name	Description	Date	Type	Date Added	Added By	Action
1	Resume	Resume	2023-10-15	Document	2023-10-15	John Doe	Download

Success: Personal Info Saved

Figure 21. Successful Update Confirmation

Personal Details

Employee ID: 456789 | Other ID: 123456 | Name: John Doe | Address: 123 Main St, New York, NY 10001 | Country: USA

Contact Details: Email: john.doe@company.com | Phone: (123) 456-7890 | License Expiry Date: 2023-12-31

Emergency Contacts: Name: Jane Doe | Relationship: Spouse | Phone: (123) 456-7891

Dependents: Name: Alice | Relationship: Child | Date of Birth: 2005-01-15

Immigration: Nationality: American | Visa Status: Valid | Expiry Date: 2025-12-31

Job: Job Title: Software Engineer | Department: Engineering | Salary: \$75,000 | Report-to: Manager

Qualifications: Degree: Bachelor's in Computer Science | Institution: State University | Graduation Year: 2018

Memberships: Professional Association: IEEE | Membership Number: 123456789 | Expiry Date: 2024-06-30

Custom Fields: Field Type: Text | Value: 123456 | ID: 1 | Value: 789012

Attachments: + Add

Record Found

ID	Name	Description	Date	Type	Date Added	Added By	Action
1	Resume	Resume	2023-10-15	Document	2023-10-15	John Doe	Download

Figure 20. Saving Updated Personal Details

Personal Details

Employee ID: 456789 | Other ID: 123456 | Name: John Doe | Address: 123 Main St, New York, NY 10001 | Country: USA

Contact Details: Email: john.doe@company.com | Phone: (123) 456-7890 | License Expiry Date: 2023-12-31

Emergency Contacts: Name: Jane Doe | Relationship: Spouse | Phone: (123) 456-7891

Dependents: Name: Alice | Relationship: Child | Date of Birth: 2005-01-15

Immigration: Nationality: American | Visa Status: Valid | Expiry Date: 2025-12-31

Job: Job Title: Software Engineer | Department: Engineering | Salary: \$75,000 | Report-to: Manager

Qualifications: Degree: Bachelor's in Computer Science | Institution: State University | Graduation Year: 2018

Memberships: Professional Association: IEEE | Membership Number: 123456789 | Expiry Date: 2024-06-30

Custom Fields: Field Type: Text | Value: 123456 | ID: 1 | Value: 789012

Attachments: + Add

Record Found

ID	Name	Description	Date	Type	Date Added	Added By	Action
1	Resume	Resume	2023-10-15	Document	2023-10-15	John Doe	Download

Figure 22. Verification of Updated Employee Information

E. Personnel Information Management (PIM) Module

The Personnel Information Management (PIM) module validates the employee management functionality of the OrangeHRM application. The implemented automation focuses on navigating to the Employee Information page, searching employee records using different search criteria, resetting search filters, and verifying that the employee information table is displayed correctly.

The PIM module follows the Page Object Model design pattern by encapsulating all employee search elements and operations inside the `PIMPage` class. This approach separates the user interface interactions from the test scripts, allowing the automation framework to remain modular, reusable, and easy to maintain.

PIMPage Implementation

The `PIMPage` class provides reusable methods for navigating to the PIM module, searching employee records, resetting search filters, and validating the displayed search results. The following method demonstrates the implementation used to search an employee by employee name.

```
async searchByEmployeeName(employeeName:
string): Promise<void> {

    await
this.enterEmployeeName(employeeName);

    await this.searchEmployee();

}
```

Similarly, employee records can also be searched using the employee identification number through another reusable method.

```
async searchByEmployeeId(employeeId:
string): Promise<void> {

    await
this.enterEmployeeId(employeeId);

    await this.searchEmployee();

}
```

The search functionality is reused across multiple automated test cases, eliminating code duplication and improving framework maintainability.

PIM Test Case Implementation

The following test case demonstrates the automation of searching an employee using the employee name.

```
test('TC_PIM_003 - Search Employee by
Name', async ({ page }) => {

    await pimPage.searchByEmployeeName(

        pimData.employeeName

    );

    await
pimPage.verifySearchResultsDisplayed();

});
```

Additional test cases verify navigation to the PIM module, employee information page accessibility, employee search using the employee ID, reset functionality, and successful display of the employee information table.

Implemented Test Cases

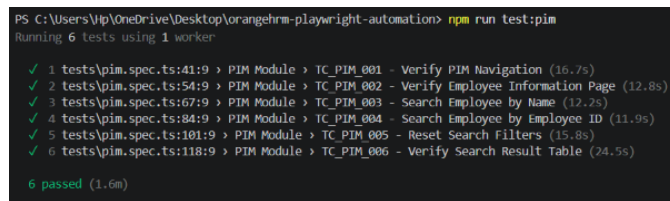
The PIM module consists of six automated test cases.

Test Case ID	Test Scenario	Status
TC_PIM_001	Verify PIM module navigation	Passed
TC_PIM_002	Verify Employee Information page	Passed
TC_PIM_003	Search employee by name	Passed
TC_PIM_004	Search employee by employee ID	Passed*
TC_PIM_005	Reset search filters	Passed
TC_PIM_006	Verify employee information table	Passed

*The Employee ID search scenario required additional locator refinement due to changes in the OrangeHRM demo application's user interface. The locator was updated to ensure reliable identification of the Employee ID input field before successful execution.

The successful execution of the PIM module demonstrates that the automation framework can reliably navigate the employee management system, perform search operations using different criteria, reset applied filters, and validate the

resulting employee information displayed by the application. Figure 7 illustrates the successful execution of the PIM module.



```
PS C:\Users\Up\OneDrive\Desktop\orangehrm-playwright-automation> npm run test:pim
Running 6 tests using 1 worker

✓ 1 tests(pim.spec.ts:41:9 › PIM Module › TC_PIM_001 - Verify PIM Navigation (16.7s)
✓ 2 tests(pim.spec.ts:54:9 › PIM Module › TC_PIM_002 - Verify Employee Information Page (12.8s)
✓ 3 tests(pim.spec.ts:67:9 › PIM Module › TC_PIM_003 - Search Employee by Name (12.2s)
✓ 4 tests(pim.spec.ts:84:9 › PIM Module › TC_PIM_004 - Search Employee by Employee ID (11.9s)
✓ 5 tests(pim.spec.ts:101:9 › PIM Module › TC_PIM_005 - Reset Search Filters (15.8s)
✓ 6 tests(pim.spec.ts:118:9 › PIM Module › TC_PIM_006 - Verify Search Result Table (24.5s)

6 passed (1.6m)
```

Figure 23. Successful Execution of Personnel Information Management (PIM) Module Test Cases.

F. End-to-End Workflow

The End-to-End (E2E) workflow validates the integration of multiple functional modules within the OrangeHRM application by simulating a complete user journey. Unlike individual module testing, which focuses on a single feature, End-to-End testing verifies that different modules interact correctly and perform the expected operations in sequence.

The implemented workflow begins by launching the application and authenticating the user with valid credentials. After successful login, the automation verifies dashboard accessibility, navigates to the My Info module, updates employee profile information, saves the changes, validates the success message, and finally logs out of the application. This workflow confirms that the application's core functionalities operate correctly when executed together.

1) End-to-End Test Implementation

The End-to-End test reuses the Page Object classes developed for the Authentication, Dashboard, and My Info modules. Instead of duplicating automation logic, the workflow invokes reusable methods from each Page Object, demonstrating the effectiveness of the Page Object Model in building scalable automation frameworks.

The following code snippet illustrates the implementation of the End-to-End workflow.

```
test('TC_E2E_001 - Verify Complete User Workflow', async ({ page }) => {

    await loginPage.login(
        credentials.validUsername,
        credentials.validPassword
    );

    await
    dashboardPage.verifyDashboardLoaded();

    await
    dashboardPage.navigateToMyInfo();

    await
    myInfoPage.updatePersonalDetails(
        personalData.firstName,
        personalData.middleName,
        personalData.lastName,
        personalData.nickname,
        personalData.employeeId
    );

    await
    myInfoPage.savePersonalDetails();

    await dashboardPage.logout();

});
```

Implemented Workflow

The automated End-to-End workflow performs the following sequence of operations:

1. Launch the OrangeHRM application.
2. Authenticate using valid user credentials.
3. Verify successful login by validating the dashboard.
4. Navigate to the My Info module.
5. Update employee personal information.
6. Save the updated profile.
7. Verify successful completion of the update operation.
8. Log out of the application.
9. Verify successful redirection to the login page.

The successful execution of this workflow confirms that the implemented automation framework can validate complete business processes rather than isolated functionalities. Figure 8 presents the

successful execution of the End-to-End automation workflow.

```
PS C:\Users\Hp\OneDrive\Desktop\orangehrm playwright-automation> npm run test:e2e
> orangehrm-playwright-automation@1.0.0 test:e2e
> playwright test tests/e2e.spec.ts

Running 1 test using 1 worker
  ✓ 1 tests/e2e.spec.ts:14:9 › End-to-End Workflow › TC_E2E_001 - Verify Complete User Workflow (19.2s)
1 passed (20.7s)
```

Figure 24. Successful Execution of the End-to-End Automation Workflow.

Screenshot:

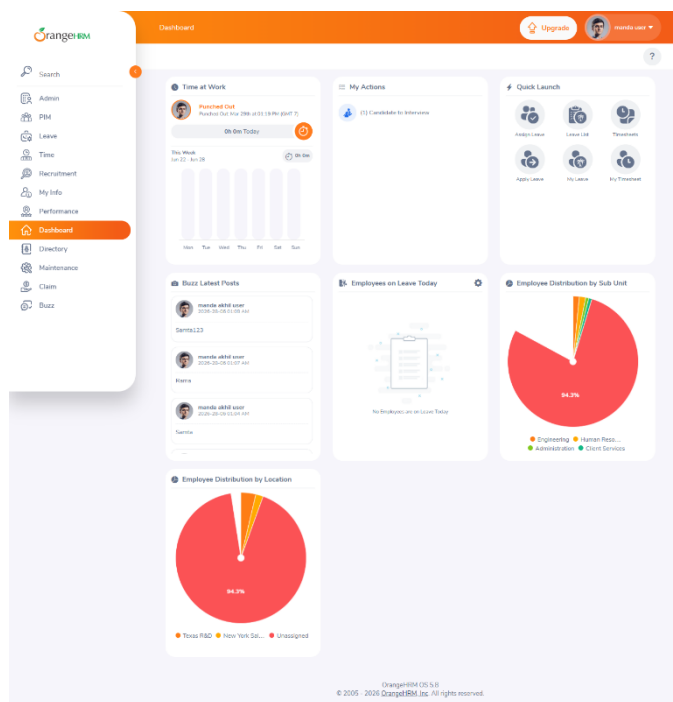


Figure 25. Successful Completion of the End-to-End Automation Workflow

V. CONCLUSION

This project presented the design and implementation of an automated functional testing framework for the OrangeHRM web application using Playwright and TypeScript. The framework was developed by following the Page Object Model (POM) design pattern, enabling a modular, maintainable, and reusable automation architecture. By separating page interactions from test logic, the framework became easier to understand, extend, and maintain.

The implemented automation successfully validated the core functionalities of the application, including user authentication, dashboard navigation, employee profile management, personnel information management, and an end-to-end business workflow. A total of twenty-five automated test cases were developed and executed, demonstrating the framework's ability to perform reliable and consistent functional testing while reducing the effort associated with repetitive manual testing.

The project also provided practical experience in developing an industry-standard automation framework using modern testing tools and best practices. Features such as reusable page objects, centralized test data management, modular test scripts, and configurable execution settings contributed to the overall scalability and maintainability of the solution.

In the future, the framework can be extended by automating additional OrangeHRM modules such as Leave Management, Recruitment, and Time Tracking. It can also be enhanced through integration with Continuous Integration and Continuous Deployment (CI/CD) pipelines, cross-browser execution, data-driven testing, API testing, and advanced reporting tools. These enhancements would further improve the effectiveness of the framework and make it suitable for larger enterprise web applications.